

Release candidate review — proto publishing workflow

Repository: <https://github.com/bondarolik/publish-workflow-demo>

Purpose

Review summary for adopting this workflow in **cm_protos**. The demo performs real GitHub operations in dry-run mode, but does not yet publish a package to CodeArtifact.

Problem to solve

- Preview a proposed contract change before it merges.
- Test approved changes together without making the testing branch part of each PR.
- Publish stable versions predictably, with an auditable version decision.
- Let downstream services choose a PR preview, QA integration build, or stable release.
- Prevent accidental releases for docs-only changes or unapproved breaking changes.

Without this workflow, version selection, QA promotion, and consumer installation are manual and can drift between teams.

One-line solution

Use GitHub Actions to validate PR-declared SemVer impact, publish tagged PR and QA previews, and release an immutable stable package from main.

Brief versioning policy

- **Git tags on main are the stable-version source of truth**; package.json is synced after a stable release.
- Each PR selects exactly one impact: **patch** (compatible fix), **minor** (compatible capability), **major** (breaking; guardian approval required), or **none** (docs/CI only; no release).
- The required **version-impact** check validates the selection, major approval, and allowed paths for none.
- Production consumers pin an exact SemVer version, for example `@lifestance/protos: 1.30.9`, rather than a moving dist-tag.

Channel	Version format	Dist-tag
PR preview	{next}-pr.{PR number}.{run}	@pr-{PR number}
QA integration	{latest}-testing.{run}	@testing
Stable	bump(latest tag, impact)	@latest

Details: [versioning policy](#) and [consumer guidance](#).

Brief GitHub Actions flow

```
Open or update a PR to main
→ version-impact validates Version Impact
→ publish-pr exposes an @pr-{PR number} preview

Add ready-for-qa
→ promote-to-testing squash-merges PR changes into testing
→ publish-testing updates @testing for integrated QA

Squash-merge the approved PR to main
```

- **publish-main** computes SemVer from latest tag + PR impact
- creates a git tag and GitHub Release
- publishes @latest once CodeArtifact publishing replaces the current dry run
- syncs package.json with the stable version

The **testing** branch is an integration environment, not an upstream branch for PRs. Resolve conflicts on testing; do not merge testing into a PR branch. After a stable release, recreate testing from main and re-promote outstanding QA PRs one at a time.

Details: [workflow reference](#), [PR flow](#), and [testing-branch procedure](#).

Troubleshooting

Symptom	Expected response
version-impact fails	Select exactly one version impact; for major, obtain the guardian label; for none, restrict changes to allowed docs/CI paths.
PR preview does not run	Ensure version-impact passed; preview publishing is intentionally skipped for none.
Promote to testing conflicts	Resolve the squash merge locally on testing, push it, and leave the PR branch unchanged.
ready-for-qa does not promote after recreating testing	Re-promote manually; an existing label does not retrigger promotion.
package.json is invalid	Remove JSON comments and do not manually edit the package version.
Stable release cannot create a tag	Check whether the tag already exists and verify the latest tag and PR impact.

Complete guide: [troubleshooting](#).

Rollout plan — cm_protos

- 1 Copy approved workflows, scripts, PR template, and policy documentation into the existing cm_protos repository.
- 2 DevOps configures CodeArtifact authentication, registry configuration, secrets or AWS role access, Actions write permissions, required labels, and main branch protection.
- 3 Implement scripts/publish-codeartifact.sh and replace the dry-run publish path.
- 4 Validate controlled PRs covering patch, minor, approved major, none, PR preview, testing promotion, and stable release.
- 5 Enable real publishing only after validation succeeds and DevOps confirms package access for consumer services.

Risks and mitigations

Risk	Mitigation
Breaking change lacks review	Major requires the PR declaration plus version:major-approved; version-impact blocks merge otherwise.
Docs-only PR publishes	The none allowlist rejects consumer-impacting files; stable publishing skips none.
QA PRs conflict or testing scope becomes unclear	Resolve only on testing; for small teams, promote and release one PR at a time, then recreate testing from main.
Production consumer changes unexpectedly	Production consumers use exact SemVer pins; pr-*, testing, and latest are controlled update channels.

Risk	Mitigation
Actions or CodeArtifact access fails	DevOps configures and validates permissions and authentication before real publishing.
Bad release reaches latest	Use the immutable git tag to identify it; consumers revert to a known-good exact pin while a corrective release is prepared.

Rollback plan

Before real publishing: Disable or revert the `cm_protos` workflow changes. The dry-run demo creates no registry state to repair.

- 1 Disable the affected GitHub Actions workflow or remove its publish permission until the issue is understood.
- 2 Do not delete or retag published stable versions; tags and package versions remain immutable audit records.
- 3 Revert consumer services to their last known-good exact `@lifescape/protos` version and commit the lockfile.
- 4 Fix the issue in a new PR, select the correct version impact, and publish a new corrective stable version.
- 5 If only testing is affected, recreate testing from main and re-promote the QA PRs that still need validation.

Approval requested

Approve adoption of this workflow in `cm_protos`, with CodeArtifact publishing enabled by implementing `scripts/publish-codeartifact.sh` in place of the existing dry-run publisher.

Source review document: https://github.com/bondarolik/publish-workflow-demo/blob/main/docs/RELEASE_CANDIDATE_REVIEW.md